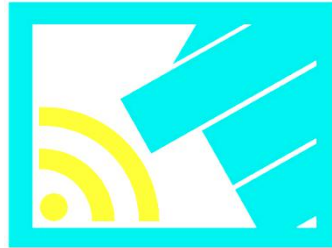


優方科技股份有限公司
Best Solution Inc.



V512 觸摸包使用說明

2022/04/15

V512

目錄

1.0 觸控參數說明.....	3
1.1 GLOBE_VARIES. INC 參數介紹.....	3
1.1.1 參數列表.....	3
1.1.2 參數說明.....	3
1.2 函數介紹(請參考*.EXT 檔的內容).....	5
1.2.1 休眠週期 RAM.....	5
1.2.2 標誌位.....	5
1.2.3 副程序.....	6
2.0 軟件包使用流程.....	10
2.1 使用流程圖.....	10
2.2 軟件包使用範例.....	11
3.0 用戶函數說明.....	14
3.1 初始化函數.....	14
3.2 工作函數.....	14
3.3 休眠函數.....	14
3.3.1 休眠函數標誌位說明.....	14
3.3.2 休眠函數定義說明.....	15
3.3.3 休眠函數使用說明.....	15
4.0 補充說明.....	16
4.1 軟件包測量電容值.....	16
4.2 使用軟件包注意事項.....	16

1.0 觸控參數說明

1.1 GLOBE_VARIES.INC 參數介紹

1.1.1 參數列表

參數	功能	數值範圍	建議值
DebounceTimes	按鍵去抖設定	0~15	2~5
AutoCalibrationPeriod	自動校準時間設定	0~15	4~8
HighSensitive	高感度設定	0=低感度 1=高感度	0
MaximumKeyHoldTime	按鍵最長反應時間	0~15	1~3
AutoFrequencyHopping	自動跳頻設定	0=關閉 ; 1=開啟	1
OneKeyActive	單鍵輸出設定	0=關閉 ; 1=開啟	視應用功能設定
PowerSave	省電功能	0=關閉 ; 1=開啟	
MovingCalibration	動態校正設定	0=正常校正 1=動態校正	1
MainFreqSelect	觸摸振蕩頻率選擇	0=3M 和 7M 1=3M 和 11M 2=1M 和 3M	默認選擇 3M 和 11M 不建議串電阻
Key Threshold	觸發閾值	10~64(十進制)	
IO_TOUCH_ATTR	觸摸按鍵/IO 功能切換	0=IO 1=KEY	
WAKEUP_KEY	按鍵喚醒使能	0=休眠時對應按鍵無法喚醒 1=休眠時對應按鍵能夠喚醒	

*以上參數均放在 GLOBE_VARIES.INC 內，可手動修改參數值

1.1.2 參數說明

DebounceTimes

設定去抖次數(值越大，按鍵反應越慢)以 10mS 為單位
(判斷_SCAN_CYCLEF 標誌位)計 1 數一次，每階增加 10 mS。當 DebounceTimes=0 時
時間最短，為 60mS-80mS。

AutoCalibrationPeriod

設定校正時間，以 80mS 為單位， 0=80 mS ; 1=160 mS……15=1280 mS 當設定的校
正時間一到，且無按鍵時，即校正一次環境(更新參考值)。

HighSensitive

設定感度高低

設為 0 時： 一般感度。

設為 1 時： 高感度。

MaximumKeyHoldTime

設定最長動作時間，以 4 秒為單位，計算最長動作時間限制

0=關閉此功能； 1=4 秒……15=60 秒。

AutoFrequencyHopping

設定硬體自動跳頻開啓或關閉。

0=關閉

1=開啓

OneKeyActive

設定單鍵輸出功能 (不支援 4KEY 及以下的 IC)

設為 0 時，按多個觸摸鍵都有對應的按鍵旗標輸出。

設為 1 時，按多個觸摸鍵只有一個鍵成立，變量最大的鍵成立，若超過 3 個(含)以上的鍵被按下，則視為所有觸摸鍵都無效。

PowerSave

設定省電模式功能

設為 0 時，關閉省電模式。

設為 1 時，無按鍵動作發生，8 秒之後自動進入休眠模式。

MovingCalibration

設定動態更新環境功能，

0=觸摸鍵按下後不更新“參考值”

1=動態更新“參考值”，不論是否有觸摸鍵成立，都依當前環境進行校正。

MainFreqSelect

設定觸摸按鍵振蕩頻率，

設為 0 時，是選擇 3M 和 7M 的觸摸按鍵振蕩頻率(可以串小於 1k 電阻)。

設為 1 時，是選擇 3M 和 11M 的觸摸按鍵振蕩頻率(不建議串電阻)。

設為 2 時，是選擇 1M 和 3M 的觸摸按鍵振蕩頻率。

Key n Threshold

設定觸發閥值，值越大，感度越低；值越小，感度越高。

建議取值為 10~64。

IO_TOUCH_ATTR

0=將對應的按鍵引腳設置成 IO 口模式。

1=將對應的按鍵引腳設置成 KEY 模式。

WAKEUP_KEY

0=對應的按鍵不可以在觸摸的時候喚醒休眠模式

1=對應的按鍵可以在觸摸的時候喚醒休眠模式

1.2 函數介紹(請參考*.EXT 檔的內容)

1.2.1 休眠週期 RAM

_STANDBY_TIME(可讀寫)

休眠週期計數器，減為 0 會進入休眠

開啟休眠模式且無按鍵時會從 127 減至 00 ($127 \times 63\text{ms} = 8 \text{ 秒}$)

如果要短時間內進入休眠，這個寄存器最小要設置為 8 ($8 \times 63\text{ms} = 0.5 \text{ 秒}$)

該值若設置低於 8 可能會出現未知狀況。

_STANDBY_TIME_CTRL(可讀寫)

休眠週期設定，取值範圍 N (8~127)，時間為 $N \times 63\text{ms}$

對該 RAM 寫值後，休眠週期計數器的最大值會設為該數值

用戶可根據實際需求設定休眠週期時間

1.2.2 標誌位

_SCAN_CYCLEF(只能讀)

所有觸摸鍵掃描一個週期標誌位，每週期約 10mS，

也是 KeyDebounce 的計時基準

_ANY_KEY_PRESSF(只能讀)

任一觸摸鍵按下標誌位

_TKS_ACTIVEF(只能讀)

觸摸軟件包完成初始化，並開始運行標誌位。

_TKS_63MSF(只能讀)

63mS 計時器標誌位。

_TKS_250MSF (只能讀)

250mS 計時器標誌位。

_TKS_500MSF(只能讀)

500mS 計時器標誌位

_FORCE_CALIBRATEF(可讀寫)

強制所有觸摸鍵，對環境做重校正標誌位

_HALT_STATE(可讀寫)

休眠判斷標誌位，根據該標誌位判斷能否進入休眠模式

1. 2. 3 副程序

_GET_KEY_BITMAP

功能：讀取“觸摸鍵”狀態；
以 bitmap 的方式輸出，相對應的 bit 為 0=放開；1 = 壓下。

輸入：無

輸出：_DATA_BUF[0]；_DATA_BUF[1] …… _DATA_BUF[N]

_DATA_BUF[0] = KEY 8 (MSB) ~ KEY1 (LSB)。

_DATA_BUF[1] = KEY16 (MSB) ~ KEY9 (LSB)。

……

_DATA_BUF[N]=KEY8*(N+1)(MSB)~KEY(8*N+1)(LSB)。

當 IC 的觸摸按鍵≤8 個時只有_DATA_BUF[0]是有效數據，IC 的觸摸按鍵≤16 個時_DATA_BUF[0]和_DATA_BUF[1]是有效數據，以 8 為一階進行類推。

範例程序：將按鍵 1~8 的按鍵狀態保存在變量 TEMP1 中，按鍵 9~16 的按鍵狀態存在變量 TEMP2 中

C 語言範例：

```
GET_KEY_BITMAP();  
TEMP1 = DATA_BUF[0];  
TEMP2 = DATA_BUF[1];
```

彙編範例：

```
CALL _GET_KEY_BITMAP  
MOV A,_DATA_BUF[0]  
MOV TEMP1,A  
MOV A,_DATA_BUF[1]  
MOV TEMP2,A
```

_GET_ENV_VALUE

功能：讀取觸摸鍵“信號”值。

輸入：ACC =0 (第 1 KEY)； 1(第 2 KEY)……N(第 N +1 KEY)

輸出：_DATA_BUF[0]

Stack：1

說明：無

範例程序：將按鍵 1 的信號值保存在變量 TEMP 中

C 語言範例：

```
_ACC = 0;  
GET_ENV_VALUE();  
TEMP = DATA_BUF[0];
```

彙編範例：

```
MOV A,0  
CALL _GET_ENV_VALUE  
MOV A,_DATA_BUF[0]  
MOV TEMP,A
```

_GET_REF_VALUE

功能：讀取觸摸鍵“參考”值。

輸入：ACC = 0 (第 1 KEY)； 1 (第 2 KEY)……19 (第 20 KEY)

輸出：_DATA_BUF[0]

Stack：1

說明：無

範例程序：將按鍵 1 的參考值保存在變量 TEMP 中

C 語言範例：

```
_ACC = 0;
GET_REF_VALUE();
TEMP = DATA_BUF[0];
```

彙編範例：

```
MOV    A,0
CALL   _GET_REF_VALUE
MOV    A,_DATA_BUF[0]
MOV    TEMP,A
```

_GET_RCC_VALUE

功能：讀取內部的平衡電容值。

輸入：ACC = 0 (第 1 KEY)； 1 (第 2 KEY)……19 (第 20 KEY)

輸出：_DATA_BUF[0]

Stack：1

說明：無

範例程序：將按鍵 1 的內建電容值保存在變量 TEMP 中

C 語言範例：

```
_ACC = 0;
GET_RCC_VALUE();
TEMP = DATA_BUF[0];
```

彙編範例：

```
MOV    A,0
CALL   _GET_RCC_VALUE
MOV    A,_DATA_BUF[0]
MOV    TEMP,A
```

_GET_LIB_VER

功能：讀取 觸摸軟件包 版本資訊。

輸入：無

輸出：_DATA_BUF[0]；_DATA_BUF[1]

Stack：1

說明：無

範例程序：獲取軟件包的版本信息並存在變量 TEMP1 與 TEMP2 中

C 語言範例：

```
GET_LIB_VER();  
TEMP1 = DATA_BUF[0];  
TEMP2 = DATA_BUF[1];
```

彙編範例：

```
CALL  _GET_LIB_VER  
MOV   A,_DATA_BUF[0]  
MOV   TEMP1,A  
MOV   A,_DATA_BUF[1]  
MOV   TEMP2,A
```

_GET_KEY_AMOUMT

功能：讀取 觸摸鍵 的總數。

輸入：無

輸出：ACC

Stack：1

說明：無

範例程序：獲取當前 IC 所有的觸摸按鍵總數並保存在變量 TEMP 中

C 語言範例：

```
GET_KEY_AMOUMT();  
TEMP = _ACC;
```

彙編範例：

```
CALL  _GET_KEY_AMOUMT  
MOV   TEMP1,ACC
```

_SET_KEY_THR

功能：設定指定按鍵閾值。

輸入：將閾值賦給 DATA_BUF[0]，

指定按鍵賦值給 ACC，0 對應 KEY1,1 對應 KEY2

輸出：無

Stack：1

說明：無

範例程序：將按鍵 1 的閾值設為變量 TEMP 中的值

C 語言範例：

```
DATA_BUF[0] = TEMP;  
_ACC = 0;  
SET_KEY_THR();
```

彙編範例：

```
MOV   A,TEMP  
MOV   _DATA_BUF[0],A  
MOV   A,0  
CALL  _SET_KEY_THR
```


LIBRARY_RESET

功能：重新平衡環境值與參考值，清除按鍵狀態位。

Stack：1

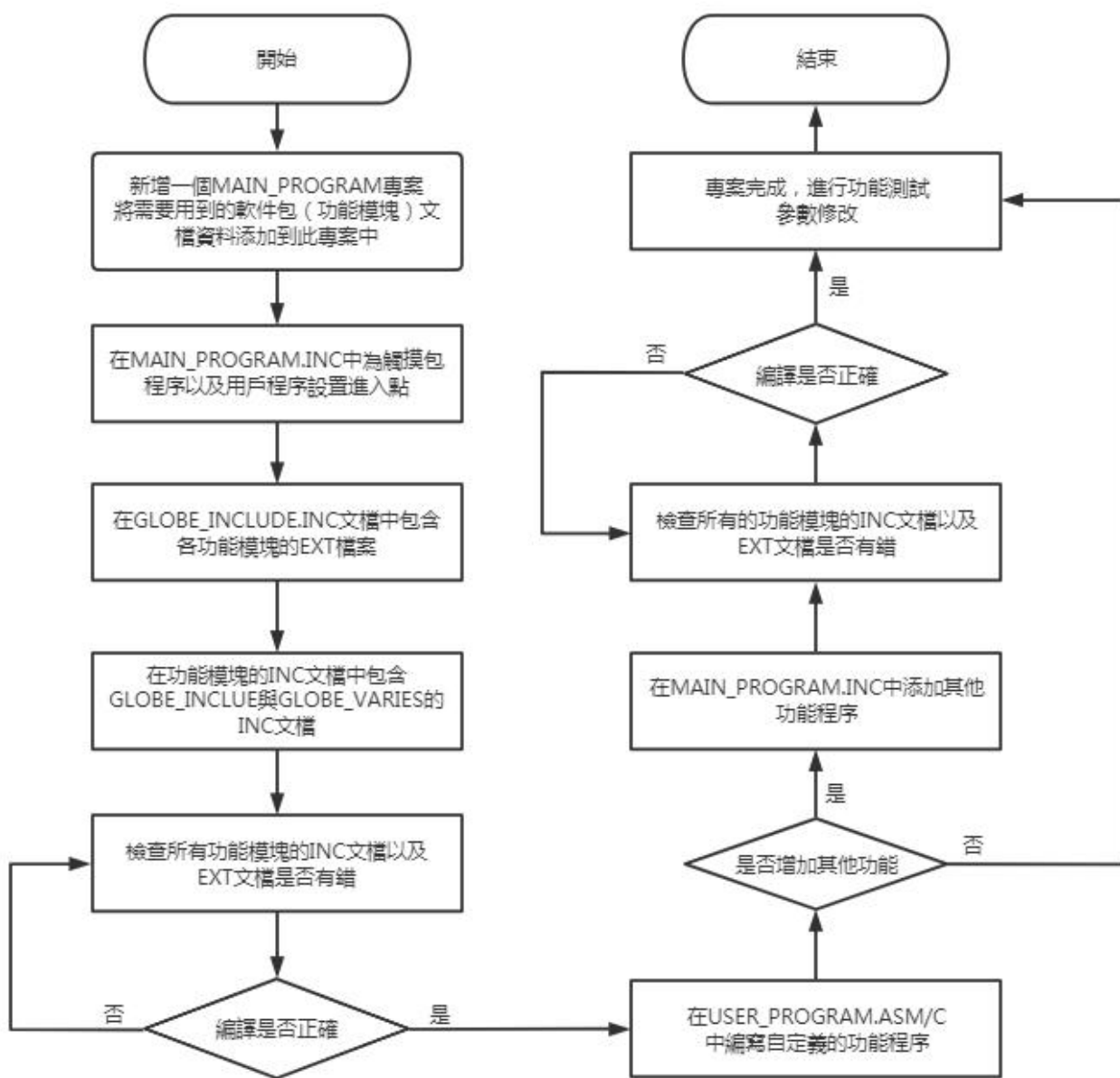
說明：無

C 語言範例：LIBRARY_RESET();

彙編範例：CALL _LIBRARY_RESET

2.0 軟件包使用流程

2.1 使用流程圖



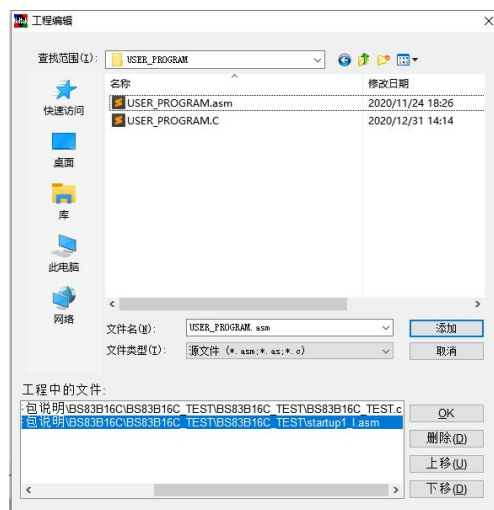
V512 軟件包將觸摸功能獨立出來，用戶可選擇是否將觸摸功能或者其他功能加入到軟件包中。上圖即為將觸摸功能與用戶程序區功能加入到軟件包後。官網下載的軟件包已將觸摸包與用戶程序功能加入，用戶可直接在 USER_PROGRAM 中編寫自己的程序。

2.2 軟件包使用範例

下面以 BS83B08C 為例，一步步講述如何設計當 IC 有任何鍵按下時，在 PA1 輸出高電平的簡單應用功能。（官網的 V512 軟件包已完成步驟一、步驟二）

步驟一：製作觸摸專案

新增一個 MAIN_PROGRAM 專案，將 BS83B08C_CTOUCH 觸摸包、USER_PROGRAM 用戶程序包的資料夾加入到軟件包後在工程/工程編輯選項中將 BS83B08C_CTOUCH 的 OBJ 檔案、USER_PROGRAM 中的用戶程序（可選擇使用彙編還是 C 語言）、以及相關的頭文件加入到專案之中。（註：任何被加進來的程序都會佔用 RAM/ROM，如感度測試程序 IIC_SLAVE，即使程序不調用也會佔用資源）



步驟二：為主函數添加觸摸包與用戶程序

將觸摸包與用戶程序的 EXT 檔案加入到專案中（EXT 檔案包含其外部函數聲明），在頭文件 GLOBE_INCLUDE.INC 中包含 EXT 檔案。在 MAIN_PROGRAM.INC 包含兩個頭文件 GLOBE_INCLUDE.INC 與 GLOBE_VARIES.INC 文檔後，為觸摸包與用戶程序設置主程序進入點，軟件包會根據 1A~2F 的順序依次執行各個類型的外部函數功能，例如當軟件包進入初始化流程時，程序會先執行 1A 的觸摸包功能後會再執行 1D 的用戶程序初始化。

```
#define MAIN_PROGRAM_DEF
#include "..\GLOBE\GLOBE_VARIES.INC"
#include "..\GLOBE\GLOBE_INCLUDE.INC"

;;;;//--FUNCTION DEFINE ;--FUNCTION NAME
#include "..\BS83B08C_CTOUCH\BS83B08C_CTOUCH.INI"
#define EXTEND_FUNCTION_1A_INITIAL _BS83B08C_CTOUCH_INITIAL
#define EXTEND_FUNCTION_1A _BS83B08C_CTOUCH
#define EXTEND_FUNCTION_1A_HALT_PREPARE _BS83B08C_CTOUCH_HALT_PREPARE
#define EXTEND_FUNCTION_1A_RETURN_MAIN _BS83B08C_CTOUCH_RETURN_MAIN

;;;;//--
//;#define EXTEND_FUNCTION_1C_INITIAL _IIC_SLAVE_INITIAL
//;#define EXTEND_FUNCTION_1C _IIC_SLAVE
//;#define EXTEND_FUNCTION_1C_HALT_PREPARE _IIC_SLAVE_HALT_PREPARE
//;#define EXTEND_FUNCTION_1C_HALT_WAKEUP _IIC_SLAVE_HALT_WAKEUP
//;#define EXTEND_FUNCTION_1C_RETURN_MAIN _IIC_SLAVE_RETURN_MAIN

;;;;//--
#define EXTEND_FUNCTION_1D_INITIAL _USER_PROGRAM_INITIAL
#define EXTEND_FUNCTION_1D _USER_PROGRAM
#define EXTEND_FUNCTION_1D_HALT_PREPARE _USER_PROGRAM_HALT_PREPARE
#define EXTEND_FUNCTION_1D_HALT_WAKEUP _USER_PROGRAM_HALT_WAKEUP
#define EXTEND_FUNCTION_1D_RETURN_MAIN _USER_PROGRAM_RETURN_MAIN
```

步驟三：設置觸摸參數

可在 GLOBE_VARIES. INC 設置軟件包的各項功能與參數，具體參數含義參考 1.1 參數介紹。

```
;;;;//-----
;;;;//--DEFINE SYSTEM CLOCK  -
;;;;//-----
#define SystemClock 0 ;;;;//0=8MHZ
; ;;;;//1=12MHZ
; ;;;;//2=16MHZ

;;;;//-----
;;;;//--TOUCH KEY LIBRARY VARIES DEFINE
;;;;//-----
#define CTOUCH_U512_DEF
;;;;//--numeric operate range 0 ~ 15;
#define DebounceTimes 0

;;;;//--numeric operate : range 0 ~ 15; // function: Time period to calibrat
#define AutoCalibrationPeriod 7 ;;;;//auto calibration period select 0

;;;;//--bit operate : function: Sensitive double up
#define HighSensitive 1 ;;;;//0=Normal ;// 1=High Sensitive

;;;;//--numeric operate ;// range 0 ~ 15: function: key holding time ,if ti
#define MaximumKeyHoldTime 2 ;;;;//0=disable ;// 1=4 second .....

;;;;//--bit operate ;// range 0/1 : function: enalbe hardware hopping fun
#define AutoFrequencyHopping 1 ;;;;//0=disable ;// 1=enable

;;;;//--bit operate ;// range 0/1 : function: only one or all key active
#define OneKeyActive 0 ;;;;//0=all key active ;// 1=one key a

;;;;//--bit operate ;// range 0/1 : function: Low power consumption
#define PowerSave 1 ;;;;//0=disable ;// 1=power save mode

;;;;//--bit operate ;// range 0/1 : function: moving calibration signal w
#define MovingCalibration 1 ;;;;//0=calibrate when key non press

;;;;//--numeric operate ;// range 0 ~ 2 : function: Main frequency selectio
#define MainFreqSelect 0 ;;;;//0:3M/7M 1:3M/11M 2:3M/1M

;;;;//--Key threshold define
#define Key1Threshold 16 ;;;;//suggestion range 10 ~ 64
#define Key2Threshold 16
#define Key3Threshold 16
#define Key4Threshold 16
;;;;//--
#define Key5Threshold 16
#define Key6Threshold 16
#define Key7Threshold 16
#define Key8Threshold 16
;;;;//--

;;;;//-----
;;;;//--DEFINE PIN AS I/O OR TOUCH INPUT
;;;;//-----
#define IO_TOUCH_ATTR 00000000000000000000000011111111B ;;;;//0=IO ;// 1=TOUCH INPUT
; ;;;;//KEY 3 ~ 2 ~ 2 ~ 1 ~ 1 ~ 0 ~ 0 ~ 0 ;//KEY32~KEY1
; ;;;;// 2 4 0 6 2 8 4 1

;;;;//-----
;;;;//--DEFINE WAKEUP KEY
;;;;//-----
#define WAKEUP_KEY 00000000000000000000000011111111B ;;;;//0=DISABLE ; 1=ENABLE
; ;;;;//KEY 3 ~ 2 ~ 2 ~ 1 ~ 1 ~ 0 ~ 0 ~ 0 ;KEY32~KEY1
; ;;;;// 2 4 0 6 2 8 4 1
```

步驟四：編譯\全部重建

若編譯提示錯誤，檢查所有功能模塊的 INC 文檔以及 EXT 是否正確。

步驟五：編寫用戶功能

在 USER_PROGRAM.ASM/C 內編寫用戶功能程序，其中 USER_PROGRAM_INITIAL() 為初始化程序，只在上電後執行一次。USER_PROGRAM() 為用戶程序區，若關閉省電模式，程序會一直執行該函數，用戶可在此編寫程序。

USER_PROGRAM.C 範例:

```
void USER_PROGRAM_INITIAL()
{
    _pac1 = 0;
}

//=====
//*****
//=====

void USER_PROGRAM()
{
    if(ANY_KEY_PRESSF)
    {
        _pa1 = 1;
    }
    else
        _pa1 = 0;
}
```

USER_PROGRAM.ASM 範例:

```
;; *****
;; * USER_PROGRAM_INITIAL *
;; *****
CLR    PAC1
RET

;; *****
;; * SUB. NAME:USER_PROGRAM *
;; * INPUT      : *
;; * OUTPUT     : *
;; * USED REG.:WORKING MODE DO THIS *
;; * FUNCTION  : *
;; *****

;; *****
;; * USER_PROGRAM: *
;; * USER PROGRAM ENTRY *
;; *****

SZ     _ANY_KEY_PRESSF
SET    PA1
SNZ    _ANY_KEY_PRESSF
CLR    PA1

RET
```

步驟六：功能測試

將檔案編譯後燒錄到 IC 中，按下任意按鍵觀察 PA1 是否輸出高電平。若功能異常可檢測上述步驟的配置是否有問題，若按鍵感度不足可以減小按鍵閾值。

3.0 用戶函數說明

3.1 初始化函數

當 IC 上電後，軟件包會依次執行在 **MIAN_PROGRAM.INC** 中定義的所有初始化函數，其中 **USER_PROGRAM_INITIAL** 為用戶初始化函數，用戶可在該函數內編寫初始化程序。正常情況下該程序只會在上電後執行一次。

3.2 工作函數

當 IC 執行完初始化後，程序會循環執行所有的外部工作函數，即循環執行底層觸摸函數與用戶寫在 **USER_PROGRAM** 中的函數。若用戶沒有開啟省電模式，軟件包會一直在工作函數間循環執行，軟件包會自動清狗並重新設置時鐘（時鐘可在 **GLOBE_VARIES.INC** 中配置），無需用戶手動清狗。

若用戶開啟了省電模式，軟件包的 **STANDBY_TIME** 會從 7FH 不斷向下計數。當計數值為 00H 後軟件包將會進入休眠流程。進入休眠前若有按鍵按下，計數值將維持在 7FH 直到無按鍵按下後重新向下計數。若用戶希望修改休眠週期可修改 **STANDBY_TIME_CTRL** 的值來實現，詳細可參考 1.2.1 休眠週期 RAM。

3.3 休眠函數

3.3.1 休眠函數標誌位說明

V512 軟件包若開啟省電模式，在 **MAIN_PROGRAM.ASM** 中會根據休眠標誌位 **HALT_STATE** 來判斷軟件包能夠進入休眠模式，在執行休眠函數前會執行 **SET HALT_STATE** 操作讓該標誌位的最高位於最低位變為 1。用戶可在休眠函數中修改 **HALT_STATE** 來對休眠流程進行控制，以下為休眠標誌位最高位與最低位的解釋說明：

HALT_STATE.bit0（可讀寫）

該位為程序能否休眠判斷位。若該位為 1 表示程序滿足休眠條件，之後會進入休眠。若該標誌位為 0 則表示程序不滿足休眠條件，之後程序會返回工作流程。

HALT_STATE.bit7（可讀寫）

該位為休眠標誌位 RAM 操作有效位。若該標誌位為 0 則視為休眠標誌位有效，軟件包會根據 **HALT_STATE.0** 決定軟件包能否進入休眠。若該標誌位為 1 則操作無效，軟件包將不會根據休眠判斷位進行流程控制。因此若要使用休眠標誌位必須將 **HALT_STATE.7** 清零。

3.3.2 休眠函數定義說明

V512 軟件包新添加了三個休眠函數供用戶使用，這三個休眠函數放在工作函數下方，這三個函數的功能分別為：

① **USER_PROGRAM_HALT_PREPARE()**

休眠準備函數：IC 進入休眠流程後執行

② **USER_PROGRAM_HALT_WAKEUP()**

休眠喚醒函數：IC 從休眠模式被中斷、IO 喚醒之後執行

③ **USER_PROGRAM_RETURN_MAIN()**

休眠返回工作函數：IC 從休眠模式返回工作模式時執行

3.3.3 休眠函數使用說明

一：USER_PROGRAM_HALT_PREPARE()

休眠前準備函數：當進入休眠週期時間減為 0 或看門狗溢出復位後會執行該函數。軟件包會依次執行所有的外部休眠前準備函數，此時觸摸包底層會進行一次按鍵掃描，即在執行該函數前會進行觸摸包底層的休眠前準備函數，判斷有無按鍵成立。

若有按鍵按下，軟件包觸摸底層函數會將休眠標誌位 HALT_STATUS 清零從而讓 IC 進入喚醒流程，若無按鍵按下則不會對 HALT_STATUS 進行操作。之後進入 USER_PROGRAM_HALT_PREPARE，此時用戶可根據實際需求在此編寫程序。

二：USER_PROGRAM_HALT_WAKEUP()

休眠喚醒函數：當 IC 進入休眠模式後被中斷或 IO 口喚醒後會執行該函數。軟件包底層函數不會修改 HALT_STATUS，滿足休眠條件。因此若用戶不在用戶休眠喚醒函數中對 HALT_STATUS 標誌位進行操作，軟件包喚醒後會繼續進入休眠模式。

若用戶希望軟件包在被中斷或 IO 口喚醒後返回工作函數，可在 USER_PROGRAM_HALT_WAKEUP 函數中將 HALT_STATUS 清零，代碼如下：

```
void USER_PROGRAM_HALT_WAKEUP( )
{
    HALT_STATUS  =  0;
}
```

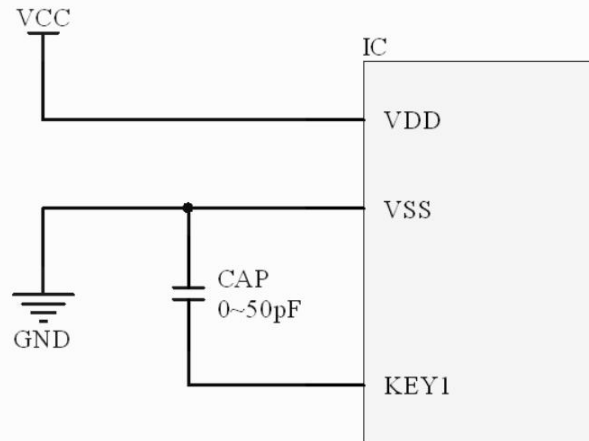
三：USER_PROGRAM_RETURN_MAIN()

休眠返回工作函數：當程序在執行完休眠前準備函數或休眠喚醒函數後，HALT_STATUS 不滿足休眠條件，軟件包會執行返回工作函數，軟件包底層在此會將所有的省電配置復原從而恢復觸摸的所有功能。用戶可在此將進入休眠前為了省電關閉的功能重新開啟。

4.0 補充說明

4.1 軟件包測量電容值

V512 軟件包支持測量 0~50pF 內的電容，只需將電容接到按鍵引腳與地中，通過高兩位低八位的內建電容值可以計算外接電容的容值，計算公式為：



$$C = \frac{\text{內建電容值} \times 50pf}{1024}$$

4.2 使用軟件包注意事項

1. 指定正確的路徑

資料夾路徑需注意是否正確，在 *.INC 或 *.H 中的 `#include "...\資料夾名稱\文檔名稱.EXT"` 有包含路徑宣告，其中 `..` 是指回到上層路徑，再指到軟件包資料夾內的 *.EXT 檔

2. 程序進入點宣告

在 MAIN_PROGRAM.INC 內，若要添加新功能如 USRT_TEST 時需要為其定義進入點，若要刪除某些功能可以將對應的程序的進入點函數注釋

3. 程序位址宣告

程序位址(含中斷位址)採絕對位址宣告，不可用相對地址 ORG 的方式宣告絕對位址：

```
PROGRAM_ENTRY    ;=====
                  .SECTION AT 000H 'CODE' ✓
                  JMP      PROGRAM_RESET
```


相對地址：

ORG	000H	X
JMP	PROGRAM_RESET	

4. 資料 RAM 宣告

資料 RAM 採 DB ; DBIT 的方式宣告

注：程序位址和資料 RAM 的宣告請參考附錄或 IDE3000 使用手冊